TP – Graded Capstone-level Assignment

ING3 - ICC – Cloud Infrastructures

Academic year 2026–2027

Introduction

The purpose of this graded assignment is to summarise what you have learnt in this cloud introductory course.

Below you will find **six assignment briefs**. Most of them must be performed on the Google Cloud Platform, although you can choose to use alternative technologies to submit a self-hosted work. Some of the proposals are tightly related to what we have seen in the course, whereas some of them are more ambitious and will require some additional learning on your side. Each assignment has: context, requirements, deliverables, grading rubric, and tips. You must choose one (and only one) assignment.

Assignment topics are just suggestions. You can decide to perform variations as long as it does not simplify the topic **and you justify the modifications**. In case of doubt, check your proposal with me.

Teams

You must work by 4-5 people teams. Before starting, please register your team in the following Google sheet:

2627-Graded project. Teams and members

Important dates

The project must be submitted no later than **Monday, December 7th 2026, at 9:00 am**.

Grading

Each assignment has its own grading rubric.

Submission and Deliverables

If your project does not use the GCP, you must submit your deliverables (diagrams, documentation, schemas, etc.) to github/gitlab and send me a link to the repository by email (juan-angel.lorenzo-del-castillo@cyu.fr).

If your project uses the GCP, you must send me by email all the required deliverables, specified on each assignment, and **add me as a project viewer** (with user juan-angel.lorenzo-del-castillo@cyu.fr), so that I can inspect your work in the GCP.

Resources

— Your GCP account.

— Free tools such as DrawIO (https://www.drawio.com) to design the architecture diagrams.

— Any Open Source platform you consider useful (e.g. Apache Airflow, Terraform, etc.)

Assignments

1 Assignment Brief 1 – Scalable Web Application Deployment

1.1 Context

You are tasked with deploying a two-tier application on Google Cloud Platform that mimics a simplified e-commerce system. The system must be scalable, secure, and cost-efficient.

1.2 Requirements

— Deploy the **frontend** using **App Engine** or **Cloud Run**.

— Deploy the **backend API** using **Cloud Run** or **Compute Engine**.

— Use **Cloud SQL** (MySQL/Postgres) for persistent storage.

— Connect frontend ⟷ backend ⟷ database securely (no hardcoded credentials).

— Expose the frontend through a public HTTPS endpoint.

1.3 Deliverables

— Architecture diagram.

— Public URLs for frontend and backend.

— Database schema with sample data.

— Step-by-step documentation.

1.4 Grading (100 pts)

— Functionality (40) – App runs correctly, DB integrated.

— Architecture (20) – Proper separation of tiers, secure connections.

— Documentation (20) – Clear steps, diagram included.

— Resource management (10) – Clever use of resources, cleans up resources.

— Innovation (10) – Extra features (auth, caching, monitoring).

1.5 Tips

— Use Cloud SQL IAM authentication.

— Start with a simple “product list” app before adding complexity.

---

2 Assignment Brief 2 – Event-Driven Data Pipeline

2.1 Context

IoT devices often send data continuously. Your task is to simulate this by creating a pipeline that ingests, processes, and stores messages in GCP.

2.2 Requirements

— Publish sample JSON messages (e.g., sensor data) into **Pub/Sub**.

— Create a **Cloud Function** (or Cloud Run service) triggered by Pub/Sub to parse and enrich messages.

— Store processed data in **BigQuery**.

— Optional: Build a dashboard with **Looker Studio**.

2.3 Deliverables

— Pub/Sub topic and subscription details.

— Cloud Function/Run code.

— BigQuery table with sample ingested data.

— Example queries (SQL).

— Documentation with architecture diagram.

2.4 Grading (100 pts)

— Functionality (40) – Messages flow through pipeline correctly.

— Architecture (20) – Event-driven, loosely coupled.

— Documentation (20) – Diagram, explanation of data flow.

— Resource mgmt (10) – Free tier usage, cleanup.

— Innovation (10) – Visualization, error handling, monitoring.

2.5 Tips

— Use JSON like `{ "device_id": 1, "temp": 22.5 }`.

— Start with one message per second using a simple script.

---

3 Assignment Brief 3 – Static Website with Global Distribution

3.1 Context

Many companies host websites with high availability and global reach. Your task is to deploy a personal portfolio/blog with CDN support.

3.2 Requirements

— Store website files in a **Cloud Storage** bucket.

— Configure bucket for static website hosting.

— Serve globally through **Cloud CDN** and **HTTPS load balancer**.

— Enable **Cloud Monitoring** for traffic and latency.

3.3 Deliverables

— Public website URL.

— Screenshots of website served through CDN.

— Monitoring dashboard screenshot.

— Documentation with architecture diagram.

3.4 Grading (100 pts)

— Functionality (40) – Website publicly accessible via HTTPS.

— Architecture (20) – Correct CDN + LB configuration.

— Documentation (20) – Clear steps, diagram.

— Resource mgmt (10) – Low-cost setup, deletion at end.

— Innovation (10) – Custom domain, advanced monitoring.

3.5 Tips

— Use free templates for your site.

— Cloud CDN may take time to propagate — plan ahead.

---

4 Assignment Brief 4 – Machine Learning Inference Service

4.1 Context

Cloud platforms allow you to deploy machine learning models at scale. Your task is to containerize a pretrained model and expose it as an API.

4.2 Requirements

— Take a small ML model (e.g., Iris classifier, sentiment analysis).

— Containerize it with **Docker**.

— Deploy container to **Cloud Run**.

— API must accept JSON input and return prediction in JSON.

— Store logs in **Cloud Logging**.

4.3 Deliverables

— Public API endpoint.

— Example input/output (e.g., curl or Postman).

— Dockerfile and code repository.

— Deployment documentation with architecture diagram.

4.4 Grading (100 pts)

— Functionality (40) – API works, returns predictions.

— Architecture (20) – Proper containerization, Cloud Run usage.

— Documentation (20) – Code repo + diagram.

— Resource mgmt (10) – Free tier usage, cleanup.

— Innovation (10) – Authentication, error handling, monitoring.

4.5 Tips

— Use scikit-learn pretrained models to keep things light.

— Avoid large datasets/models to control costs.

---

5 Assignment Brief 5 – DevOps CI/CD on GCP

5.1 Context

Modern cloud apps are deployed via CI/CD pipelines. Your task is to set up an automated deployment for a simple app.

5.2 Requirements

— Host source code on **Cloud Source Repositories** or GitHub.

— Configure **Cloud Build** pipeline to build container image.

— Deploy image automatically to **Cloud Run**.

— Test versioning and rollback.

5.3 Deliverables

— Cloud Build YAML config.

— Repo URL (if public) with sample code.

— Screenshots/logs of build and deployment.

— Documentation with architecture diagram.

5.4 Grading (100 pts)

— Functionality (40) – Pipeline builds and deploys correctly.

— Architecture (20) – Proper CI/CD workflow.

— Documentation (20) – Clear pipeline explanation, diagram.

— Resource mgmt (10) – Free tier usage, cleanup.

— Innovation (10) – Automated tests, rollback demo.

5.5 Tips

— Start with a "Hello World" container to test pipeline.

— Use triggers for push-to-main branch deployments.

---

6 Assignment Brief 6 – Container Orchestration with Kubernetes (GKE)

6.1 Context

Many companies deploy applications using **containers** orchestrated by Kubernetes. Your task is to containerize a simple web application and deploy it on **Google Kubernetes Engine (GKE)**, demonstrating scalability and resiliency.

6.2 Requirements

— Containerize a simple web app (e.g., Python Flask, Node.js Express, or similar).

— Push the image to **Artifact Registry** (or Docker Hub).

— Create a **GKE cluster** (small size, e.g., 2 nodes).

— Deploy the app using Kubernetes manifests (`Deployment`, `Service`).

— Expose it via a **LoadBalancer** service with an external IP.

— Demonstrate **scaling** by increasing replicas.

6.3 Deliverables

— Public URL of the deployed app.

— Kubernetes manifests (YAML files).

— Architecture diagram (cluster, pods, services).

— Documentation including steps to build, deploy, and scale.

6.4 Grading (100 pts)

— Functionality (40) – App runs correctly, accessible externally.

— Architecture (20) – Proper use of Deployments, Services, and scaling.

— Documentation (20) – Clear instructions, YAML manifests, diagram.

— Resource mgmt (10) – Minimal cluster size, free credits, cleanup.

— Innovation (10) – Use of ConfigMaps, Secrets, monitoring, or autoscaling.

6.5 Tips

— Use `kubectl scale deployment` to test manual scaling.

— Keep the app lightweight (static website, simple API) to avoid costs.
